

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT I
DEEP LEARNING AND ITS APPLICATIONS — CO3133
FOUNDATIONS OF DEEP LEARNING PIPELINES AND
ARCHITECTURES

From Linear Models to Modern Sequence Models: A
Comparative Study for Image Classification

MAJOR: COMPUTER SCIENCE

CLASS: CC04

SUPERVISOR: PhD. Lê Thành Sách

— o0o —

STUDENT: HOÀNG XUÂN BÁCH 2352082

NGUYỄN VIỆT HÙNG 2352424

LÊ NGUYỄN GIA PHÚC 2352931

HO CHI MINH CITY, September 2026

Contents

1	Problem and Data Description	1
1.1	Problem formulation (Hung)	1
1.1.1	Real-world motivation and ideal outcome	1
1.1.2	Machine-learning task	1
1.1.3	Mathematical formulation	2
1.1.4	Input representation across model families	2
1.1.5	Research question and comparison objective	3
1.1.6	Success criteria and evaluation metrics	3
1.1.7	Constraints, assumptions, and main difficulties	4
1.1.8	Scope statement	4
1.2	Dataset description (Bach)	4
1.2.1	Datasets used in this assignment	4
1.2.2	Source, version, and license	5
1.2.3	Collection and labeling pipeline	5
1.2.4	Sample counts and annotation format	5
1.2.5	Storage size and memory footprint	5
1.2.6	Dataset bias and limitations	6
1.3	Exploratory Data Analysis (Bach)	6
1.3.1	Input dimensionality and integrity	6
1.3.2	Class balance and distribution analysis	7
1.3.3	Pixel-intensity distribution and normalization rationale	7
1.3.4	Qualitative inspection of representative samples	9
1.3.5	Summary of EDA insights and connection to data splitting	9
1.4	Data splitting (Phuc)	11
1.4.1	Dataset Partitioning and Split Strategy	11
1.4.2	Seed and Deterministic Reproducibility	11
1.4.3	Stratification	12
1.4.4	Split Unit	12
1.4.5	Data Leakage Prevention	12
1.4.6	Subset Selection Rules and Fairness Constraint	13
1.5	Preprocessing (Phuc)	13
2	Methodology	14

2.1	Pipeline Overview (Phuc)	14
2.2	Baseline (Linear / softmax classifier)	14
2.2.1	Architecture and input representation	14
2.2.2	Loss and evaluation metrics	14
2.2.3	Parameter count	15
2.2.4	Strengths and limitations	15
2.2.5	Self-implemented components and library dependencies	16
2.3	Main methods	16
2.3.1	Multilayer Perceptron (MLP) (Bách)	16
2.3.2	Convolutional Neural Network (CNN) (Phúc)	16
2.4	Convolutional Neural Network (CNN) Architecture	16
2.4.1	Architecture Description	16
2.4.2	Input Representation	17
2.4.3	Loss Function	17
2.4.4	Evaluation Metrics	17
2.4.5	Parameter Budget Analysis	18
2.4.6	Strengths and Limitations	18
2.4.7	Implementation Provenance	19
3	Training setup	20
3.0.1	Detailed Protocol Specifications	20
4	Experimental design	22
5	Implementation Results	23
5.1	Baseline (Linear / softmax classifier) (Hung)	23
5.1.1	Training behavior	23
5.1.2	Quantitative results	23
5.1.3	Qualitative results	23
5.2	Multilayer Perceptron (MLP) (Bách)	23
5.2.1	Training behavior	23
5.2.2	Quantitative results	23
5.2.3	Qualitative results	23
5.3	Convolutional Neural Network (CNN) (Phúc)	23
5.3.1	Training behavior	23
5.3.2	Quantitative results	23

5.3.3	Qualitative results	23
5.4	Comparison and discussion	23
5.5	Error analysis	23
5.6	Ablation study	23
5.7	Limitation	23
5.8	Conclusion	23

List of Figures

1	Verified sample count per class across the official training and test partitions.	7
2	Log-scale pixel intensity distribution across the training partition, highlighting the 50.2% pure black background spike at zero.	8
3	Six randomly sampled training images per class for classes 0–4 (T-shirt/top, Trouser, Pullover, Dress, Coat; seed 42).	9
4	Six randomly sampled training images per class for classes 5–9 (Sandal, Shirt, Sneaker, Bag, Ankle boot; seed 42).	10

List of Tables

1	Core elements of the machine-learning problem.	1
2	Input representation and architectural inductive bias.	2
3	Evaluation evidence required for the controlled comparison.	3
4	Fashion-MNIST class mapping and verified sample counts.	6
5	Mean pixel intensity per class across the official training partition.	8
6	Dataset Partitioning and Class Distribution Summary	11
7	Layer-by-layer Trainable Parameter Breakdown of SimpleCNN	18
8	Standardized Training and Optimization Hyperparameters	20

1. Problem and Data Description

1.1. Problem formulation (Hung)

1.1.1. Real-world motivation and ideal outcome

Clothing-image recognition is a compact but meaningful example of visual perception. A reliable classifier must distinguish visually similar products—for example, a shirt from a T-shirt/top or a pullover from a coat—from low-resolution grayscale images. The ideal outcome of this assignment is not merely to obtain the highest test accuracy. It is to understand how different neural-network families process the same visual evidence and to identify the trade-offs among predictive quality, computational cost, and inductive bias (the assumptions built into a model architecture).

The corresponding model goal is:

Given one Fashion-MNIST image, predict exactly one of its ten clothing categories. Train and compare five required architecture families under a common data split and evaluation protocol so that observed differences can be attributed as fairly as possible to the model architecture and input representation.

1.1.2. Machine-learning task

This is a predictive, multiclass single-label classification problem. *Multiclass* means that more than two categories are possible, while *single-label* means that each image has exactly one ground-truth category. No proxy label is required: the dataset’s clothing-category label directly represents the target to be predicted.

Table 1: Core elements of the machine-learning problem.

Element	Definition
Prediction unit	One individual Fashion-MNIST image.
Input	A 28×28 grayscale clothing image, represented after preprocessing as a tensor $\mathbf{x} \in \mathbb{R}^{1 \times 28 \times 28}$.
Output	Ten real-valued logits, one per clothing class; the predicted label is the class with the largest logit.
Target	One integer class label $y \in \{0, 1, \dots, 9\}$.
Primary dataset	Fashion-MNIST, with MNIST restricted to development/debugging and CIFAR-10 optional and outside the core comparison.
Required models	Linear/softmax classifier, MLP, CNN, LSTM or GRU, and Transformer.

The ten target categories are T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The prediction is made from image pixels only; metadata or information from other samples must not be used as an input to the classifier.

1.1.3. Mathematical formulation

Let the labeled dataset be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)_{i=1}^N\}, \quad \mathbf{x}_i \in \mathcal{X} \subseteq \mathbb{R}^{1 \times 28 \times 28}, \quad y_i \in \mathcal{Y} = \{0, 1, \dots, 9\}. \quad (1)$$

A model parameterized by θ maps an image to ten logits:

$$f_\theta : \mathcal{X} \rightarrow \mathbb{R}^{10}, \quad \mathbf{z}_i = f_\theta(\mathbf{x}_i). \quad (2)$$

The logits are converted into class probabilities by

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=0}^9 \exp(z_{i,k})}, \quad c \in \mathcal{Y}, \quad (3)$$

and the predicted class is

$$\hat{y}_i = \arg \max_{c \in \mathcal{Y}} p_{i,c} = \arg \max_{c \in \mathcal{Y}} z_{i,c}. \quad (4)$$

All five models are trained by minimizing empirical cross-entropy risk on the training set:

$$\theta^* = \arg \min_{\theta} \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (-\log p_{i,y_i}). \quad (5)$$

In PyTorch, `CrossEntropyLoss` receives the raw logits $\mathbf{z}_i$ and internally applies the required log-softmax operation. Therefore, an explicit softmax layer must *not* be applied before this loss.

1.1.4. Input representation across model families

The semantic task remains unchanged, but each architecture receives a representation suited to its structure. This controlled variation is central to the comparison.

Table 2: Input representation and architectural inductive bias.

Model	Input representation	Main inductive bias
Linear	Flattened vector in $\mathbb{R}^{784}$	Linear decision boundaries; no explicit spatial structure.
MLP	Flattened vector in $\mathbb{R}^{784}$	Nonlinear global feature interactions, but no explicit locality.
CNN	Tensor in $\mathbb{R}^{1 \times 28 \times 28}$	Local connectivity and shared filters preserve two-dimensional spatial structure.
LSTM/GRU	Sequence of 28 rows (or columns), each in $\mathbb{R}^{28}$	Sequential dependence and a recurrent hidden state summarize preceding timesteps.
Transformer	Sequence of projected rows, columns, or image patches with positional encoding	Self-attention models pairwise token relationships; positional encoding supplies ordering information.

1.1.5. Research question and comparison objective

The main research question is:

How do input representation and architectural inductive bias affect classification quality, efficiency, and error patterns when linear, feed-forward, convolutional, recurrent, and attention-based models are trained on the same Fashion-MNIST split?

We expect the CNN to provide a strong trade-off between quality and efficiency because it explicitly models local spatial patterns and shares parameters across image locations. The Linear model serves as a low-capacity baseline; the MLP tests the benefit of nonlinear feature learning; and the LSTM/GRU and Transformer test whether treating a two-dimensional image as a sequence is competitive under the same controlled setting. These are hypotheses to be tested, not conclusions assumed in advance.

1.1.6. Success criteria and evaluation metrics

Following the framing guideline, assignment-level success criteria are separated from model evaluation metrics. The study is considered successful if it:

1. produces a reproducible end-to-end PyTorch pipeline for all five mandatory models;
2. compares them using the same train/validation/test partitions, evaluation protocol, random seed policy, and checkpoint-selection rule;
3. explains performance using both quantitative evidence and qualitative error analysis rather than selecting a model from accuracy alone; and
4. identifies defensible trade-offs among predictive performance, parameter count, training time, inference time, representation, and inductive bias.

The mandatory model-level evidence is summarized in Table 3.

Table 3: Evaluation evidence required for the controlled comparison.

Evidence	Purpose
Accuracy	Measures the overall proportion of correct predictions.
Macro-F1	Computes F1 for each class and averages classes equally, revealing whether performance is consistent across categories.
Parameter count	Describes model size and capacity.
Training and inference time	Measures computational cost and practical efficiency.
Training/validation curves	Diagnose convergence, underfitting, and overfitting.
Confusion matrix	Reveals which clothing categories are systematically confused.
Correct/incorrect examples	Supports qualitative inspection of easy, difficult, and failed predictions.

1.1.7. Constraints, assumptions, and main difficulties

- **Fairness of comparison:** All core models must use the same dataset split and metrics. Any model-specific preprocessing or augmentation must be disclosed and justified because it may confound the architecture comparison.
- **Data leakage prevention:** Test data must not influence training, hyperparameter selection, early stopping, normalization statistics, or checkpoint selection. The test set is reserved for final evaluation.
- **Reproducibility:** The report must state sample counts per split, random seeds, hardware, software versions, training configuration, and checkpoint rule.
- **Low resolution and visual ambiguity:** Fashion-MNIST images contain limited detail, while several classes have similar silhouettes. This can create systematic confusions such as Shirt versus T-shirt/top or Pullover versus Coat.
- **Representation mismatch:** Flattening removes explicit two-dimensional neighborhood structure, whereas row/column sequences impose an artificial scan order. These choices may advantage or disadvantage particular architectures.
- **Efficiency versus quality:** a more complex model may improve a metric but require substantially more parameters or computation; therefore, “best” must be interpreted as a multi-criteria judgment.
- **Scope limitation:** conclusions are specific to a small grayscale benchmark and should not automatically be generalized to high-resolution, color, or real-world production imagery.

1.1.8. Scope statement

The core experiment reports primary results on Fashion-MNIST and includes all five mandatory architecture families. MNIST may be used only to verify code correctness during development. CIFAR-10, pretrained backbones, robustness tests, interpretability methods, and matched-compute studies are optional extensions and cannot substitute for missing core requirements.

1.2. Dataset description (Bach)

1.2.1. Datasets used in this assignment

Following the course requirements, two benchmarks from the MNIST family are considered. MNIST (handwritten digits, 10 classes) is strictly reserved for initial pipeline development, unit sanity checks, and debugging (DataLoader behavior, forward-backward step integrity, and evaluation metrics) prior to running larger compute jobs. Fashion-MNIST serves as the primary dataset for all comparative experimental results presented in this report. All five mandatory model architectures—Linear/Softmax

classifier, Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Gated Recurrent Unit (GRU)/LSTM, and Vision Transformer—are trained, validated, and benchmarked on Fashion-MNIST under a strictly identical data split and evaluation protocol.

1.2.2. Source, version, and license

Fashion-MNIST was introduced in 2017 by Zalando Research (Han Xiao, Kashif Rasul, and Roland Vollgraf) as a direct drop-in replacement for the original MNIST digit dataset. It was designed to provide a more challenging classification benchmark while retaining the exact 28×28 grayscale image dimensionality, 10-class cardinality, and 60,000/10,000 train/test split. The dataset is distributed under the MIT License (Copyright © 2017 Zalando SE) and is hosted publicly at <https://github.com/zalando-research/fashion-mnist>. In our experimental pipeline, we pull the official release directly via `torchvision.datasets.FashionMNIST`, reading the canonical IDX byte files.

1.2.3. Collection and labeling pipeline

The dataset comprises single-channel grayscale e-commerce product photos sourced from Zalando’s online retail catalog. The original high-resolution, full-color studio product images underwent automated background removal, aspect-ratio-preserving resizing, centering, and downsampling to 28×28 pixels. Each sample is associated with exactly one ground-truth label among 10 mutually exclusive garment categories directly mapped from Zalando’s product categorization hierarchy. No secondary manual re-annotation or label curation was performed by our team.

1.2.4. Sample counts and annotation format

Fashion-MNIST contains a total of 70,000 samples, divided by default into an official training partition of 60,000 images and an official test partition of 10,000 images. Each image is paired with a scalar target $y \in \{0, 1, \dots, 9\}$. We verified these counts empirically via direct programmatic inspection of the decompressed byte arrays. The class mappings and verified counts are summarized in Table 4.

1.2.5. Storage size and memory footprint

When downloaded and extracted by `torchvision`, the raw directory on disk occupies approximately 85.8 megabytes; in memory, each sample is initially represented as a 28×28 `uint8` array with pixel intensities ranging in $[0, 255]$. Upon ingestion into the PyTorch pipeline via `ToTensor`, images are cast to single-precision floating-point tensors with dimensions $1 \times 28 \times 28$ and values scaled to $[0.0, 1.0]$. The memory footprint of the full 60,000 training images in `float32` format is approximately $60,000 \times 28 \times 28 \times 4$ bytes ≈ 188.16 MB.

Table 4: Fashion-MNIST class mapping and verified sample counts.

Label	Class Name	Official Train	Official Test
0	T-shirt/top	6,000	1,000
1	Trouser	6,000	1,000
2	Pullover	6,000	1,000
3	Dress	6,000	1,000
4	Coat	6,000	1,000
5	Sandal	6,000	1,000
6	Shirt	6,000	1,000
7	Sneaker	6,000	1,000
8	Bag	6,000	1,000
9	Ankle boot	6,000	1,000
Total		60,000	10,000

1.2.6. Dataset bias and limitations

Several structural constraints of Fashion-MNIST must be accounted for during modeling:

- **Commercial catalog bias:** Images originate exclusively from studio photography with uniform lighting, centered bounding boxes, and isolated frontal/side perspectives. Consequently, models trained on this benchmark do not automatically generalize to in-the-wild consumer photos characterized by arbitrary angles, cluttered backgrounds, and lighting variations.
- **Silhouette ambiguity at low resolution:** At 28×28 resolution, fine-grained details such as stitching, fabric weave, buttons, and collars are suppressed. In particular, upper-body garment categories (*T-shirt/top*, *Shirt*, *Pullover*, *Coat*) present substantial geometric overlap in silhouette, forming an intrinsic bottleneck for linear and shallow classifiers.
- **Synthetic class balance:** Because the raw dataset is artificially curated to be perfectly uniform (10.0% per class), natural retail prevalence skew is absent. Performance degradation on specific classes is therefore attributable strictly to visual ambiguity rather than class starvation.

1.3. Exploratory Data Analysis (Bach)

All descriptive statistics and visualizations below were generated directly from our experimental pipeline notebook (`Assignment1_EDA.ipynb`), computing metrics over the raw tensors without downsampling.

1.3.1. Input dimensionality and integrity

Every sample across both training and test sets measures exactly $28 \times 28 \times 1$ pixels. No corrupted files, missing entries, or dimension mismatches exist. This spatial consistency ensures that all five candidate architectures ingest the exact same spatial grid, varying solely in how they structure inductive bias (spatial convolutions, recurrence over rows, sequential token attention, or spatial flattening).

1.3.2. Class balance and distribution analysis

Figure 1 displays the sample distribution across all 10 classes for both splits. Our programmatic check found a class-count standard deviation of zero in both partitions, confirming that the dataset is perfectly balanced: exactly 6,000 training images and 1,000 test images per category.

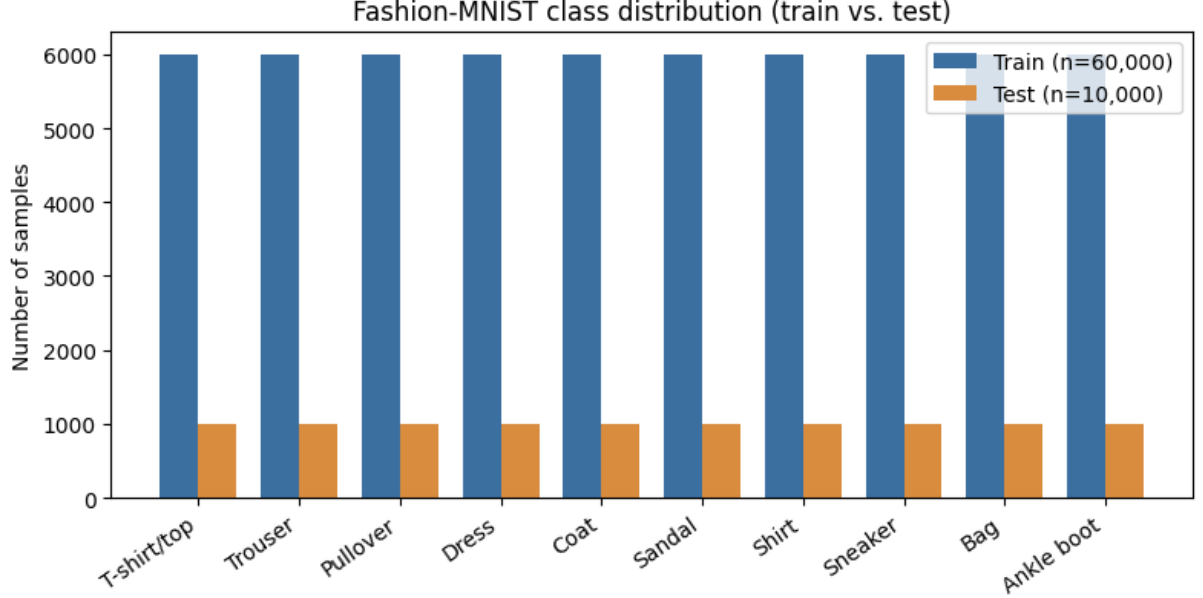


Figure 1: Verified sample count per class across the official training and test partitions.

The implications for our evaluation framework are:

- **Headline metric validity:** Top-1 overall accuracy remains a mathematically sound primary metric because the majority-class baseline is trivial (10.0%). Nonetheless, to fulfill course rigor and monitor per-class recall discrepancies on overlapping categories, we report macro-averaged F1 alongside full confusion matrices.
- **Isolated inductive comparison:** Any observed inter-class performance disparity (e.g., lower F1 score on *Shirt*) can be definitively attributed to representation difficulty rather than training sample scarcity.

1.3.3. Pixel-intensity distribution and normalization rationale

Over all $60,000 \times 28 \times 28 = 47,040,000$ pixels of the raw training partition, the global intensity statistics are:

$$\mu_{\text{raw}} = 72.94, \quad \sigma_{\text{raw}} = 90.02 \quad (6)$$

As shown in the log-scale intensity histogram in Figure 2, the distribution is prominently bimodal. Exactly 50.2% of all pixels evaluate to zero, representing the uniform black studio background. The remaining mass spans $[1, 255]$, corresponding to the garment body, folds, and surface textures.

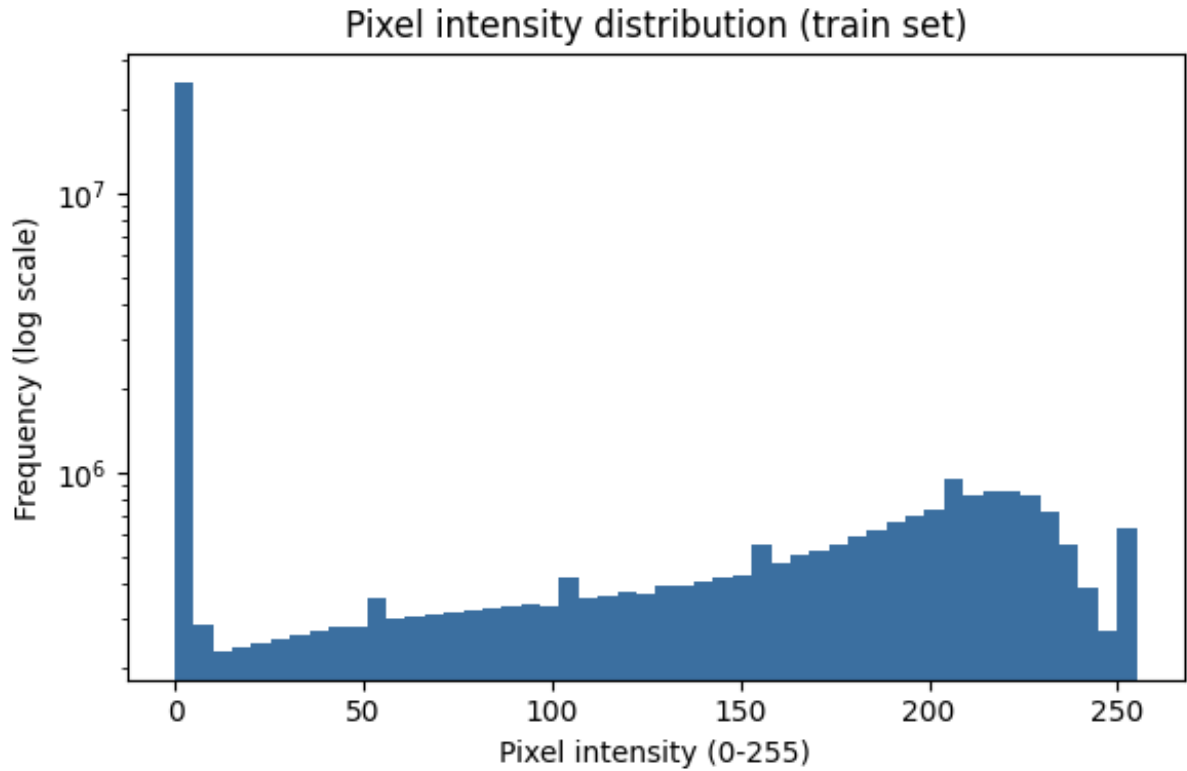


Figure 2: Log-scale pixel intensity distribution across the training partition, highlighting the 50.2% pure black background spike at zero.

Table 5 details the mean pixel brightness per category, which acts as a direct proxy for average silhouette area and garment bulkiness. Footwear categories (*Sandal*, *Sneaker*) display the lowest average intensities (34.87 and 42.76) due to large surrounding margins of empty background. Conversely, heavy outer garments (*Pullover*, *Coat*) occupy most of the frame, yielding average intensities of 96.06 and 98.26. This confirms that total foreground area provides an informative low-level cue that even simple linear models can exploit to separate footwear from apparel.

Table 5: Mean pixel intensity per class across the official training partition.

Garment Category	Mean Pixel Intensity [0, 255]
Sandal	34.87
Sneaker	42.76
Trouser	56.84
Dress	66.02
Ankle boot	76.81
T-shirt/top	83.03
Shirt	84.61
Bag	90.16
Pullover	96.06
Coat	98.26

1.3.4. Qualitative inspection of representative samples

To visually inspect intra-class variability and boundary ambiguity, Figures 3 and 4 display six randomly selected training images per class (fixed seed 42).

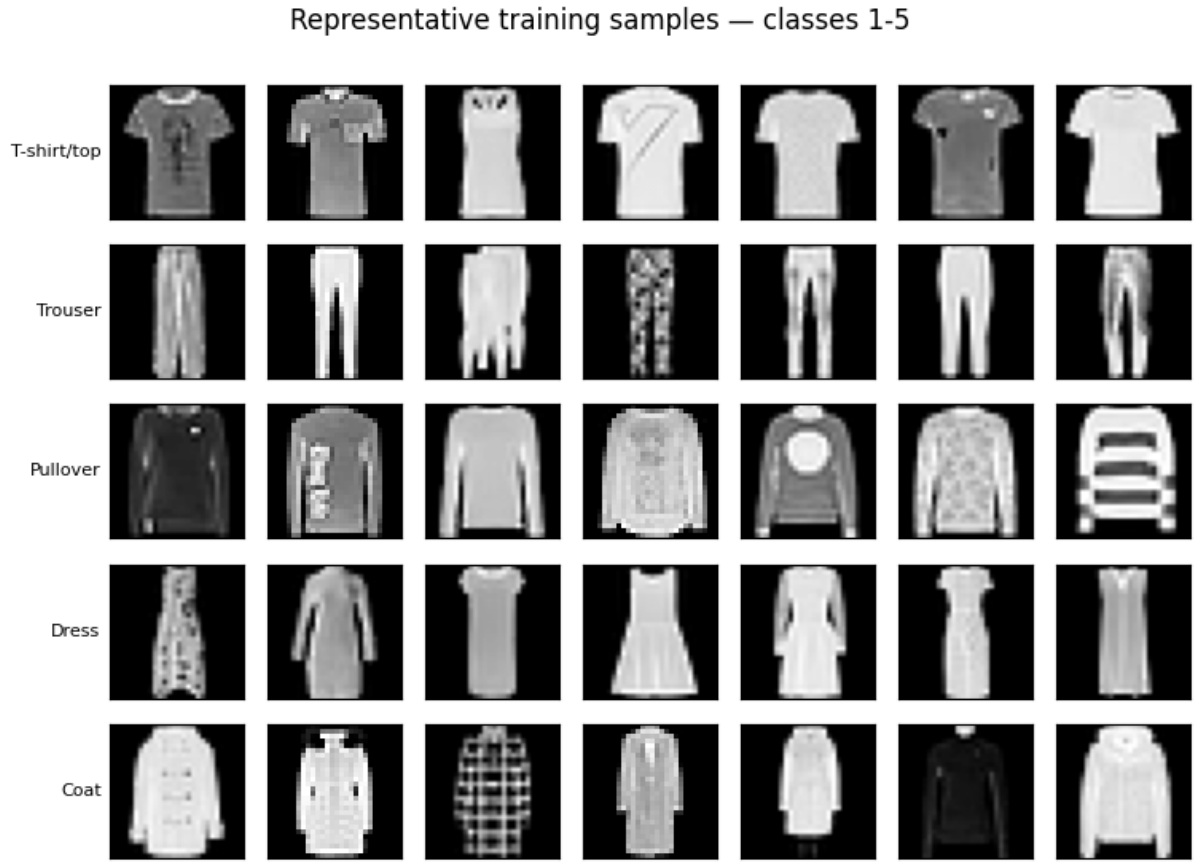


Figure 3: Six randomly sampled training images per class for classes 0–4 (T-shirt/top, Trouser, Pullover, Dress, Coat; seed 42).

The qualitative analysis reveals:

- **Substantial intra-class variance:** Classes such as *Bag* encompass handbags, backpacks, and clutches of varying structural shapes and strap orientations, requiring nonlinear boundary partitioning.
- **Visual overlap in upper-body clothing:** Distinguishing between *T-shirt/top*, *Shirt*, *Pullover*, and *Coat* is hindered by low contrast and lack of color/material depth, pinpointing this cluster as the primary source of anticipated confusion matrix error.

1.3.5. Summary of EDA insights and connection to data splitting

- **Preprocessing requirement:** Due to zero-pixel background dominance, standardizing by empirical channel statistics (μ, σ) or scaling to $[0, 1]$ prevents gradient instability while preserving structural silhouettes.

Representative training samples — classes 6-10



Figure 4: Six randomly sampled training images per class for classes 5–9 (Sandal, Shirt, Sneaker, Bag, Ankle boot; seed 42).

- **Cross-architecture compatibility:** Uniform dimensions (28×28) permit identical tensor transformations across models without spatial cropping or aspect-ratio distortion.
- **Validation partition protocol (resolved TODO):** As detailed in Section 1.4, to enforce strict leakage prevention, the official 60,000-image training pool is partitioned into 48,000 training images (80%) and 12,000 validation images (20%) using stratified sampling with global seed `SEED = 42`. This stratification ensures that both subsets maintain exact uniform distributions (exactly 4,800 train and 1,200 validation samples per class), while the 10,000 official test samples remain completely untouched. Normalization statistics (μ, σ) are computed exclusively on the 48,000 training samples.

1.4. Data splitting (Phuc)

1.4.1. Dataset Partitioning and Split Strategy

The complete Fashion-MNIST dataset comprises 70,000 grayscale images of size 28×28 distributed across 10 distinct categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). We partition the data into three disjoint subsets combining official and custom splits:

- **Official Test Split:** We preserve the 10,000 test images provided by the official Fashion-MNIST benchmark through `torchvision.datasets.FashionMNIST` with `train=False`.
- **Custom Train / Validation Split:** The official 60,000-image training pool is divided into train (80%) and validation (20%) sets using `sklearn.model_selection.train_test_split`:
 - **Training set:** 48,000 images
 - **Validation set:** 12,000 images

Table 6: Dataset Partitioning and Class Distribution Summary

Subset	Total Samples	Samples / Class
Train (Custom 80%)	48,000	4,800
Validation (Custom 20%)	12,000	1,200
Test (Official)	10,000	1,000
Total	70,000	7,000

1.4.2. Seed and Deterministic Reproducibility

A global random seed is fixed across all environments:

$$\text{SEED} = 42 \tag{7}$$

The value 42 is assigned consistently to Python’s internal PRNG, NumPy, the PyTorch CPU and CUDA generators, and the split generator. This guarantees exact repeatability of dataset splits and mini-batch sequences.

1.4.3. Stratification

Stratified sampling is applied when splitting the official 60,000 training images into the final training set (48,000 images) and validation set (12,000 images). Since the original 60,000 training images contain exactly 6,000 samples per class, stratification maintains an exact uniform distribution in both sets:

- **Training set:** exactly 4,800 images for each class
- **Validation set:** exactly 1,200 images for each class

This ensures that neither the training set nor the validation set suffers from class imbalance or sampling bias.

1.4.4. Split Unit

The elementary split unit is a **single independent image** (*one image = one sample*). Each sample corresponds to an independently cataloged e-commerce product. There are no hierarchical groupings, multi-view captures, subject/patient identities, or temporal dependencies that could introduce inter-sample correlation.

1.4.5. Data Leakage Prevention

To prevent any statistical contamination or optimistic bias from the validation/test sets leaking into the training pipeline, the following measures are enforced:

1. **Split-Before-Processing Order:** Data splitting is performed directly on raw uint8 arrays before calculating any dataset-level statistics or applying transformations.
2. **Isolated Normalization Statistics:** The pixel mean (μ) and standard deviation (σ) are estimated **exclusively from the 48,000 training images**:

$$\mu = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_i, \quad \sigma = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_i - \mu)^2} \quad (8)$$

These statistics are then frozen and applied to the validation and test sets for Z-score normalization. No validation or test pixels contribute to μ or σ .

3. **Data Augmentation Isolation:** Stochastic horizontal flipping, rotation, cropping, and color jitter are applied solely to the training split. Validation and test loaders apply only the deterministic transforms ToTensor and Normalize, with `shuffle=False`.

4. **Held-Out Test Isolation:** The official test set is completely isolated during training. It is never used for hyperparameter tuning, model selection, or early stopping decisions (which are driven purely by validation loss).

1.4.6. Subset Selection Rules and Fairness Constraint

- **Debug Subset (MNIST):** An auxiliary MNIST dataset is loaded early in the pipeline solely for sanity checks and pipeline verification; it is strictly excluded from all reported tables and figures.
- **Fairness Constraint:** all five mandatory architectures (Linear, MLP, CNN, GRU, and Transformer) are trained, validated, and evaluated on the exact same indices (`train_idx`, `val_idx`) and identical `DataLoader` configurations. Because the data and training setup are identical, any difference in performance comes directly from how each model is designed, not from variations in the data.

1.5. Preprocessing (Phuc)

2. Methodology

2.1. Pipeline Overview (Phuc)

2.2. Baseline (Linear / softmax classifier)

2.2.1. Architecture and input representation

We use a linear softmax classifier, also known as multinomial logistic regression, as the baseline for Fashion-MNIST classification. This model provides a low-complexity reference for assessing the benefits of nonlinear feature learning in the MLP and spatial feature extraction in the CNN.

Each preprocessed grayscale image has shape $1 \times 28 \times 28$. The model flattens this tensor into a vector $\mathbf{x} \in \mathbb{R}^{784}$, preserving all pixel values in a fixed order. Flattening does not discard pixel values, but the classifier does not explicitly exploit two-dimensional neighborhoods or share weights across spatial locations.

The architecture consists of a single fully connected layer:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (9)$$

where $\mathbf{W} \in \mathbb{R}^{10 \times 784}$, $\mathbf{b} \in \mathbb{R}^{10}$, and $\mathbf{z} \in \mathbb{R}^{10}$ contains one logit per class. There are no hidden layers, nonlinear hidden activations, convolutional operations, or dropout layers. The baseline uses the same image preprocessing and data partitions as the other models.

Class probabilities are conceptually obtained through softmax:

$$p_c = \frac{\exp(z_c)}{\sum_{k=0}^9 \exp(z_k)}, \quad c \in \{0, \dots, 9\}. \quad (10)$$

The predicted label is

$$\hat{y} = \arg \max_{c \in \{0, \dots, 9\}} z_c = \arg \max_{c \in \{0, \dots, 9\}} p_c. \quad (11)$$

Therefore, explicit probability computation is unnecessary when only the predicted class is required.

2.2.2. Loss and evaluation metrics

The classifier is trained using multiclass cross-entropy loss. For a mini-batch of B examples, the objective is

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \left(\frac{\exp(z_{i,y_i})}{\sum_{c=0}^9 \exp(z_{i,c})} \right), \quad (12)$$

where y_i denotes the ground-truth class of example i .

In the implementation, raw logits are passed directly to PyTorch's `nn.CrossEntropyLoss`, which in-

corporate log-softmax and negative log-likelihood. Accordingly, the forward method does not apply an additional softmax layer before the loss.

Predictive performance is evaluated using overall accuracy and macro-averaged precision, recall, and F1-score. Macro-F1 is calculated by computing F1 separately for each class and averaging the ten scores equally:

$$F_{1,\text{macro}} = \frac{1}{10} \sum_{c=0}^9 F_{1,c}. \quad (13)$$

Per-class metrics and a confusion matrix support the analysis of category-specific errors. Training and validation loss curves describe learning behavior, while parameter count, training time, and inference time characterize computational cost. Validation loss controls checkpoint selection and early stopping under the shared training protocol.

2.2.3. *Parameter count*

The fully connected layer contains 784×10 weights and ten bias terms:

$$N_{\text{parameters}} = \underbrace{784 \times 10}_{\text{weights}} + \underbrace{10}_{\text{biases}} = \boxed{7,850}. \quad (14)$$

All 7,850 parameters are trainable. Flattening, softmax, and the loss function introduce no additional trainable parameters.

2.2.4. *Strengths and limitations*

2.2.4.1 Strengths. The baseline has a compact architecture, low storage requirements, and a low arithmetic cost per prediction. Its class-specific weights can also be reshaped into 28×28 maps to inspect how individual pixel locations contribute to class scores. It provides a useful reference for determining how much classification performance can be obtained directly from weighted pixel intensities.

2.2.4.2 Limitations. Its main limitation is restricted representational capacity. Although softmax produces nonlinear probabilities, the decision boundary between any two classes remains linear in the input vector. The model cannot learn nonlinear combinations of local patterns or hierarchical visual features. Its position-specific weights also provide no built-in translation invariance, making it potentially sensitive to shifts and rotations. These limitations can hinder discrimination between visually similar clothing categories. However, the observed performance also depends on preprocessing and optimization; it should not be interpreted as an absolute upper bound for all linear classifiers.

2.2.5. Self-implemented components and library dependencies

We implemented the `LinearClassifier` wrapper, its input-flattening and forward-pass logic, the custom `ArrayImageDataset`, and the orchestration of training, validation, early stopping, checkpoint selection, and timing. These components were written using library primitives rather than implementing numerical operations or differentiation from scratch.

The main library dependencies are:

- **PyTorch:** Provides `nn.Module`, `nn.Linear`, `nn.CrossEntropyLoss`, automatic differentiation, the Adam optimizer, the `ReduceLROnPlateau` scheduler, and `DataLoader`.
- **torchvision:** Provides the Fashion-MNIST dataset interface and preprocessing/augmentation operations.
- **Pillow:** Converts image arrays to PIL images for preprocessing.
- **NumPy:** Supports array handling and prediction aggregation.
- **scikit-learn:** Provides stratified splitting and classification metrics, including the confusion matrix.
- **pandas and Matplotlib:** Provide result tables and visualizations, respectively.

No pretrained weights or external pretrained backbone are used.

2.3. Main methods

2.3.1. Multilayer Perceptron (MLP) (Bách)

2.3.2. Convolutional Neural Network (CNN) (Phúc)

2.4. Convolutional Neural Network (CNN) Architecture

2.4.1. Architecture Description

The CNN is designed following a modular hierarchical feature extraction paradigm composed of two primary sub-networks: a convolutional representation learner (features) and a classification head (classifier).

- **Feature Extraction Stage (features):**
 - **Stage 1:** `Conv2D(1 → 64, kernel = 3 × 3, padding = 1) → BatchNorm2d(64) → ReLU → MaxPool2d(2 × 2, stride = 2)`. Downsamples spatial dimensions from 28×28 to 14×14 .
 - **Stage 2:** `Conv2D(64 → 64, kernel = 3 × 3, padding = 1) → BatchNorm2d(64) → ReLU → MaxPool2d(2 × 2, stride = 2)`. Downsamples spatial dimensions from 14×14 to 7×7 .

– **Stage 3:** Conv2D($64 \rightarrow 128$, kernel = 3×3 , padding = 1) $\rightarrow$ BatchNorm2d(128) $\rightarrow$ ReLU. Operates on 7×7 feature maps to learn deeper semantic combinations without further spatial reduction.

- **Classification Stage (classifier):** Rather than flattening the high-dimensional feature tensor ($7 \times 7 \times 128 = 6,272$ dimensions), which would induce parameter explosion and severe overfitting, the network applies **Global Average Pooling** (AdaptiveAvgPool2D(1×1)). This collapses each spatial channel into a single scalar, yielding a compact 128-dimensional vector. This is followed by a Flatten layer, a regularizing Dropout layer ($p = 0.3$), and a single linear projection Linear($128 \rightarrow 10$) producing raw unnormalized logits for the 10 fashion classes.

2.4.2. Input Representation

The model receives a 4-dimensional tensor batch $\mathbf{X} \in \mathbb{R}^{B \times 1 \times 28 \times 28}$ with data type `torch.float32`. Unlike linear and multi-layer perceptron models that flatten images into 1D vectors, the CNN explicitly preserves the 2D lattice geometry of the pixel grid. Input pixel intensities $x \in [0, 255]$ are rescaled to $[0.0, 1.0]$ and standardized via Z-score normalization:

$$\tilde{x} = \frac{x - \mu}{\sigma}, \quad \text{where } \mu = 0.2860, \sigma = 0.3530 \quad (15)$$

The statistics μ and σ are estimated strictly over the 48,000 training images to prevent data snooping and information leakage.

2.4.3. Loss Function

The model is optimized using the multi-class Cross-Entropy loss on raw logits $\mathbf{z} \in \mathbb{R}^C$ ($C = 10$):

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{B} \sum_{i=1}^B \log \left(\frac{\exp(z_{y^{(i)}})}{\sum_{j=0}^{C-1} \exp(z_j)} \right) \quad (16)$$

where $y^{(i)}$ denotes the ground-truth categorical label for the i -th instance. In our implementation via PyTorch's `nn.CrossEntropyLoss`, the Softmax transformation and negative log-likelihood are mathematically fused into a single kernel, ensuring numerical stability and preventing gradient underflow.

2.4.4. Evaluation Metrics

Model performance is evaluated comprehensively across three orthogonal dimensions:

1. **Macro Classification Metrics:** Test Accuracy (86.95%), Macro-averaged F1-Score (0.8680), Macro-Precision (0.8691), and Macro-Recall (0.8695). Macro-averaging computes unweighted arithmetic means across all classes, penalizing models that sacrifice difficult categories to boost

majority-class scores.

2. **Granular Per-Class Assessment:** Independent Precision, Recall, and F1-Scores computed across each individual class $c \in \{0, \dots, 9\}$, coupled with full Confusion Matrix visualization.
3. **Computational & Runtime Profile:** Total trainable parameters (113,226), total training time (1,240.1 seconds over 45 epochs with Early Stopping), and GPU inference latency (0.1644 ms/sample, measured with CUDA synchronization and warmup).

2.4.5. Parameter Budget Analysis

To enable rigorous, controlled benchmarking against the Multi-Layer Perceptron (MLP) architecture under a matched compute/parameter constraint, the channel dimensions were systematically tuned to power-of-two values (64, 64, 128). As summarized in Table 7, the network comprises exactly 113,226 trainable parameters, differing from the comparable MLP (113,546 parameters) by merely 320 weights (0.28%).

Table 7: Layer-by-layer Trainable Parameter Breakdown of SimpleCNN

Block / Layer	Configuration	Parameter Formula	Count
Conv Block 1	Conv2D($1 \rightarrow 64, 3 \times 3, \text{pad} = 1$)	$(64 \times 1 \times 3 \times 3) + 64$	640
	BatchNorm2d(64)	$\gamma(64) + \beta(64)$	128
Conv Block 2	Conv2D($64 \rightarrow 64, 3 \times 3, \text{pad} = 1$)	$(64 \times 64 \times 3 \times 3) + 64$	36,928
	BatchNorm2d(64)	$\gamma(64) + \beta(64)$	128
Conv Block 3	Conv2D($64 \rightarrow 128, 3 \times 3, \text{pad} = 1$)	$(128 \times 64 \times 3 \times 3) + 128$	73,856
	BatchNorm2d(128)	$\gamma(128) + \beta(128)$	256
Classifier Head	AdaptiveAvgPool2D(1×1)	Non-parametric	0
	Dropout($p = 0.3$)	Non-parametric	0
	Linear($128 \rightarrow 10$)	$(10 \times 128) + 10$	1,290
Total Trainable			113,226

2.4.6. Strengths and Limitations

Strengths:

- **Spatial Inductive Bias:** By enforcing *Local Receptive Fields* and *Weight Sharing*, the convolutional kernels achieve translation equivariance, learning edge, texture, and contour detectors that generalize across arbitrary pixel coordinates.
- **Hierarchical Feature Extraction:** Progressing from low-level geometric primitives (Stage 1) to composite structural attributes such as sleeves, soles, and collars (Stage 3).
- **Extreme Parameter Efficiency via GAP:** The Global Average Pooling mechanism reduces classifier parameters to only 1,290 (1.1% of total capacity), concentrating 98.9% of the parameter budget on representation learning while mitigating downstream overfitting.
- **Rapid Convergence:** Achieves optimal validation loss at epoch 45, converging 37.5% faster than

the parameter-matched MLP (72 epochs).

Limitations:

- **Bounded Receptive Field:** Small 3×3 kernels cannot capture non-local, long-range dependencies in early stages without successive downsampling operations.
- **Inter-Class Visual Ambiguity:** The model exhibits structural vulnerabilities on fine-grained clothing categories with high morphological overlap. Specifically, class `Shirt` achieves an F1-score of only 0.6004 (Recall 0.5440) due to severe confusion with `T-shirt/top` and `Coat`, where distinguishing features (e.g., collar stiffness, buttons) are degraded by the low 28×28 grayscale resolution.
- **Rigid Geometric Sensitivity:** Standard convolutions lack intrinsic rotation invariance, requiring explicit data augmentations (`RandomRotation`, `RandomHorizontalFlip`) to maintain robustness against orientation shifts.

2.4.7. Implementation Provenance

In compliance with reproducible research guidelines, we state the origin of all components:

- **Self-Implemented Components:**

1. The entire modular network class `SimpleCNN(nn.Module)`, including the staging of convolutional blocks, pooling strides, channel expansion factors, and global average pooling classifier head.
2. The core training and validation pipeline functions: `fit()`, `evaluate()`, `run_one_epoch()`, and `train_model()`, incorporating validation loss tracking, early stopping (*patience* = 5), and automatic checkpoint serialization (`best_val_loss`).
3. Benchmarking protocols: `measure_inference_time()` featuring CUDA synchronization barriers (`torch.cuda.synchronize`) and GPU warmup iterations.
4. The granular metric aggregation suite: `compute_class_metrics()` compiling per-class performance tables and overall macro averages.

- **Library and Framework Dependencies:**

1. **PyTorch** (`torch.nn`, **version 2.14.0+cu126**): Atomic neural primitives including `Conv2d`, `BatchNorm2d`, `ReLU`, `MaxPool2d`, `AdaptiveAvgPool2d`, `Dropout`, `Linear`, and loss computation via `CrossEntropyLoss`.
2. **PyTorch Optimization** (`torch.optim`): The Adam optimizer and dynamic learning rate

scheduler ReduceLROnPlateau.

3. **TorchVision** (`torchvision.datasets`, `torchvision.transforms`, **version 0.29.0+cu126**): Automated acquisition of the raw FashionMNIST dataset and image preprocessing pipelines (`RandomHorizontalFlip`, `RandomRotation`, `ColorJitter`, `ToTensor`, `Normalize`).
4. **Scikit-Learn** (`sklearn`, **version 2.2.3**): Stratified dataset splitting via `train_test_split`, and numerical evaluation routines (`precision_score`, `recall_score`, `f1_score`, `confusion_matrix`).
5. **Data Handling and Visualization**: `pandas` (v2.2.3) for structured metrics export and `matplotlib` (v3.10.0) for training curve and confusion matrix generation.

3. Training setup

To ensure complete fairness, reproducibility, and rigorous comparative analysis across all candidate architectures, an identical and strictly standardized training protocol was enforced. Table 8 summarizes the exact hyperparameter configurations and experimental environment.

Table 8: Standardized Training and Optimization Hyperparameters

Hyperparameter / Component	Specification / Value
Optimization Algorithm	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$)
Base Learning Rate (η)	5×10^{-4}
Batch Size	512 (Train, Validation, and Test)
Maximum Epochs	100 (with Early Stopping)
Learning Rate Scheduler	ReduceLROnPlateau (factor = 0.5, patience = 1, mode = ‘min’)
Regularization	Dropout ($p = 0.3$), Batch Normalization, Data Augmentation
Early Stopping Criterion	Validation Loss (patience = 5 epochs, $\Delta_{\min} = 10^{-4}$)
Model Checkpoint Criterion	Lowest Validation Loss ($\min \mathcal{L}_{\text{val}}$)
Random Seed	42 (Global synchronization across Python, NumPy, PyTorch)
Hardware Accelerator	NVIDIA GeForce RTX 3050 Laptop GPU (6GB VRAM, CUDA 12.6)
Host Environment	x86_64 CPU, 16GB Host RAM, Windows 11 OS
Numerical Precision	Full Precision (FP32, <code>torch.float32</code>)
CNN Training Wall-Clock Time	1,240.1 seconds (45 epochs, ~ 27.56 s/epoch)
Hyperparameter Tuning	Controlled Heuristic Grid Search on Validation Split

3.0.1. Detailed Protocol Specifications

1. **Optimizer:** Optimization is governed by the **Adam** (Adaptive Moment Estimation) algorithm. Adam computes individual adaptive learning rates for disparate parameters using exponentially decaying averages of past gradients (m_t) and past squared gradients (v_t). Default moment decay coefficients ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and numerical stabilization denominator $\epsilon = 10^{-8}$ were maintained to guarantee smooth convergence across both convolutional and dense layers.
2. **Learning Rate and Scheduler:** An initial learning rate of $\eta_0 = 5 \times 10^{-4}$ was established. To facilitate fine-grained convergence into narrow loss basins without oscillation around local minima,

training dynamically employs the ReduceLROnPlateau scheduler:

$$\eta_{e+1} = \begin{cases} 0.5 \times \eta_e, & \text{if } \mathcal{L}_{\text{val}}^{(e)} \geq \min_{k < e} \mathcal{L}_{\text{val}}^{(k)} - 10^{-4} \text{ for 1 epoch} \\ \eta_e, & \text{otherwise} \end{cases} \quad (17)$$

Whenever validation loss exhibits stagnation for one complete epoch, the learning rate is halved, enabling the model to refine its weights at progressively smaller step sizes before early stopping triggers.

3. **Batch Size and GPU Utilization:** A mini-batch size of $B = 512$ is selected. For the 48,000-sample training set, this yields exactly 94 optimization steps per epoch. This mini-batch size maximizes the compute saturation of the RTX 3050 Tensor/CUDA cores, shifting execution from memory-bandwidth latency to dense compute throughput, while dampening extreme stochastic gradient oscillations.
4. **Epoch Budget and Early Stopping:** Although a ceiling of 100 epochs is allocated, execution is bounded by an automated **Early Stopping** callback. Training terminates if the validation loss fails to decrease by at least $\Delta_{\min} = 10^{-4}$ over 5 consecutive epochs (patience = 5). This prevents empirical over-fitting and guards against wasteful compute expenditure once validation performance plateaus.
5. **Checkpointing Criterion:** Model serialization follows a strict *best-validation-loss* criterion. At the end of each epoch, if $\mathcal{L}_{\text{val}}$ achieves a new global minimum, an in-memory deep copy of the network state dictionary (`model.state_dict()`) is captured. Upon training termination, the optimal checkpoint is restored and serialized to persistent storage (`CNN.pt`), ensuring downstream evaluation on the held-out test set reflects peak generalization capability rather than the final (potentially overfitted) state.
6. **Regularization Suite:** Overfitting is controlled via a multi-tiered defense:
 - *Stochastic Dropout:* A dropout rate of $p = 0.3$ is positioned prior to the linear output projection, preventing co-adaptation of high-level latent representations.
 - *Batch Normalization:* Incorporated within every convolutional block (BatchNorm2D), normalizing layer inputs to zero mean and unit variance. This reduces internal covariate shift and exerts a mild stochastic regularizing effect.
 - *Data Augmentation:* Training pipelines apply label-preserving spatial transforms on the fly: `RandomHorizontalFlip(p = 0.5)`, `RandomRotation( $\pm 10^\circ$ )`, and `ColorJitter(brightness`

= 0.2, contrast = 0.2). Vertical flipping is intentionally excluded to preserve intrinsic garment morphology.

7. **Reproducibility and Seed Protocol:** Global deterministic execution is enforced by fixing SEED = 42 at the initial runtime entry point. The seed is propagated systematically across Python’s built-in random, numpy.random, torch.manual_seed, and torch.cuda.manual_seed_all. Furthermore, the stratified 80/20 train/validation split utilizes this exact seed, ensuring identical data partitions across all benchmarked architectures.
8. **Hardware and Mixed Precision Policy:** All experiments were executed locally on an NVIDIA GeForce RTX 3050 Laptop GPU (6GB GDDR6 VRAM, CUDA capability 8.6, CUDA Driver 12.6). **Mixed precision (FP16/AMP) was explicitly omitted** in favor of full single-precision floating-point arithmetic (torch.float32). This decision eliminates numerical instability, gradient scaling artifacts, and underflow risks in lightweight layers, thereby preserving exact bitwise gradient reproducibility.
9. **Training Wall-Clock Time:** Under the specified hardware and batch parameters, the CNN completed 45 epochs before triggering early stopping, totaling 1,240.1 **seconds** (≈ 20.67 minutes, averaging 27.56 seconds per epoch). In comparison, the parameter-matched MLP required 1,916.4 seconds across 72 epochs, highlighting the superior convergence efficiency of the convolutional architecture.
10. **Hyperparameter Selection Methodology:** Hyperparameters were established via a disciplined, iterative validation-split exploration:
 - The base learning rate was evaluated over the logarithmic grid $\eta \in \{10^{-2}, 10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$. While 10^{-3} caused premature divergence in initial epochs, 5×10^{-4} demonstrated optimal gradient stability and minimal loss variance.
 - The batch size was calibrated between 128, 256, and 512; 512 delivered the highest throughput without degrading validation accuracy.
 - The fairness constraint mandated that once the optimizer family, learning rate, scheduler, and early-stopping thresholds were locked, they remained invariant across all competing models.

4. Experimental design

5. Implementation Results

5.1. Baseline (Linear / softmax classifier) (Hung)

5.1.1. *Training behavior*

5.1.2. *Quantitative results*

5.1.3. *Qualitative results*

5.2. Multilayer Perceptron (MLP) (Bách)

5.2.1. *Training behavior*

5.2.2. *Quantitative results*

5.2.3. *Qualitative results*

5.3. Convolutional Neural Network (CNN) (Phúc)

5.3.1. *Training behavior*

5.3.2. *Quantitative results*

5.3.3. *Qualitative results*

5.4. Comparison and discussion

5.5. Error analysis

5.6. Ablation study

5.7. Limitation

5.8. Conclusion